



TECHNICAL PAPER - TP-002

Flattening Oracle Nested Tables for Cloud Migration

Engineering Patterns for Modernizing Oracle 19c Object-Relational
Structures
for Amazon Aurora MySQL and Aurora PostgreSQL

FINAL PUBLICATION EDITION

BeyondX LLC | Engineering Publication | September 2026

Technology with Purpose. Opportunity with Impact.
Imagination to Implementation.

beyondxllc.com

© 2026 BeyondX LLC. All rights reserved.

Abstract

Oracle object-relational features can create a portability barrier when an Oracle 19c workload is prepared for a cloud-native relational target. This paper presents a methodology for transforming Abstract Data Types (ADTs) and single-level nested collections into conventional relational structures while preserving business meaning, relationships, integrity controls, access paths, application behavior, and rollback capability. The case study covers approximately 52 affected source tables and approximately 3.6 TB of nested-table data. The approach combines structural classification, dependency analysis, three relational flattening patterns, transitional fallback synchronization, layered validation, and explicit retirement criteria. The central conclusion is that flattening should be treated as a pre-migration modernization workstream rather than a datatype-conversion exercise.

Keywords: Oracle 19c, ADT, nested tables, object-relational modernization, flattening, Amazon Aurora MySQL, Amazon Aurora PostgreSQL, schema modernization, fallback synchronization, validation

Executive Summary

Migration from Oracle Database to a cloud-native relational platform can require significant modernization before data movement begins. Oracle Abstract Data Types (ADTs) and nested tables are examples of object-relational structures that do not have a direct, semantically equivalent implementation in Amazon Aurora MySQL-Compatible Edition or Amazon Aurora PostgreSQL-Compatible Edition.

This paper presents engineering patterns developed during a large-scale Oracle Database 19c Enterprise Edition modernization involving approximately 52 source tables and approximately 3.6 TB of object-relational and nested-table data. The source environment used ADTs and single-level (Level-1) nested tables; no multi-level nested tables were present in the case study.

The work demonstrated that flattening is not simply a datatype conversion. It is a controlled database-modernization workstream requiring structural discovery, dependency analysis, relational redesign, integrity preservation, synchronization, reconciliation, application testing, and cutover planning.

Modernization should preserve business meaning and required behavior - not necessarily the physical implementation of the legacy database.

Four engineering patterns are emphasized in this paper:

- Pattern A - Inline Flattening: place a small, non-repeating ADT into relational columns of the parent table when appropriate.
- Pattern B - One-to-One Extension Flattening: place a larger ADT in a separate 1:1 extension table when inline expansion would make the parent excessively wide.
- Pattern C - One-to-Many Collection Flattening: transform a repeating Level-1 nested collection into a relational child table, carrying the parent key as the relationship key and introducing a stable child identity where appropriate.

- Pattern D - Transitional Fallback Synchronization: temporarily synchronize legacy and flattened representations during an agreed fallback window.

The methodology distinguishes evaluated approaches from production-oriented design principles, including incremental synchronization, duplicate protection, transaction-safe trigger/procedure interaction, explicit dependency remediation, and validation beyond row counts.

1. Purpose and Scope

The purpose of TP-002 is to provide a methodology-first engineering guide for organizations that must remove Oracle-specific object-relational structures before migrating to a cloud-native relational database. It is not intended to reproduce a client implementation or provide a drop-in script library.

All schema, table, column, application, and program names in this paper are fictional. SQL fragments are illustrative reference implementations. They demonstrate patterns and decision logic, not exact production code.

Publication note. The paper intentionally emphasizes transferable architecture, controls, and validation practices rather than client-specific implementation details or development history.

2. Case-Study Context

The engineering patterns were developed during preparation of a large Oracle 19c Enterprise Edition environment for migration to a cloud-native relational platform. Approximately 52 tables used Oracle object-relational structures, representing approximately 3.6 TB of nested data.

- Source platform: Oracle Database 19c Enterprise Edition.
- Object-relational structures: ADTs and Level-1 nested tables.
- No multi-level nested tables were present in the case-study scope.
- Target design considerations included Amazon Aurora MySQL-Compatible Edition and Amazon Aurora PostgreSQL-Compatible Edition.
- A controlled fallback period of approximately two weeks was required.

The engineering objective was to preserve data relationships, business rules, application behavior, integrity controls, important access paths, and operational recoverability while eliminating Oracle-specific structures that would complicate the final migration.

3. Why Flattening Was a Pre-Migration Requirement

A complex database migration is often better understood as two transformations rather than one: first, modernization of proprietary structures into portable relational structures; second, movement of those relational structures to the target cloud database. This separation reduces the number of platform-specific behaviors that must be solved during the migration window itself.

Oracle-specific implementation -> portable relational implementation -> cloud migration

Flattening therefore belongs in the pre-migration modernization phase. It changes the schema contract, and potentially the database code and application SQL that depend on that contract. Treating it as a data-copy exercise understates the engineering risk.

4. Discovery Before Transformation

Discovery is the first control point. For each affected structure, the engineering team should inventory the object or collection type, nesting level, parent key, element cardinality, data volume, nullability, constraints, indexes, dependent database objects, application DML, and fallback requirements.

Discovery Area	Questions to Resolve
Structure	Is this an ADT, a Level-1 nested collection, or another object-relational construct?
Identity	What identifies the parent? Does each collection element have a stable business or surrogate key?
Integrity	Which CHECK constraints, foreign keys, null rules, and uniqueness rules carry business meaning?
Access paths	Which indexes and predicates support critical queries?
Dependencies	Which triggers, PL/SQL units, jobs/events, views, and application SQL reference the structure?
Operations	How will initial population, synchronization, fallback, validation, and retirement be controlled?

5. Dependency Discovery and Impact Analysis

Object-relational structures rarely exist in isolation. The modernization effort therefore included analysis of dependent triggers, PL/SQL procedures and packages, scheduled jobs or events, views, constraints, foreign keys, indexes, and application SQL/DML. Each dependency was assessed for modification, rewrite, replacement, or retirement.

This dependency analysis should remain focused on behavior rather than implementation history. The publication-level question is not which individual script was tried first; it is which business or operational behavior must continue after the data model changes.

Dependency	Analysis Focus	Typical Mitigation
Triggers	Reads/writes ADT or collection data; firing order; concurrency	Rewrite against flattened columns/child tables; validate DML behavior
PL/SQL	Object attributes, TABLE() access, collection operators	Rewrite SQL/PLSQL for relational representation; regression test

Jobs / events	Periodic processing of legacy structures	Modify SQL and schedule logic; prevent obsolete post-cutover processing
Views	Hidden ADT attributes or collection expansion	Rewrite while preserving the consumer contract where practical
Constraints / FKs	Business rules and relationships	Recreate or redesign on flattened structures
Indexes	Legacy predicates and access paths	Design indexes for new joins, filters, and relationship keys
Application SQL / DML	Queries and write patterns	Remediate statements and perform application-level testing

A database structure cannot be modernized safely without understanding the code and database objects that depend on it.

6. Classification and Decision Framework

Classification converts discovery into a repeatable design decision. The case study used two primary source categories - ADTs and Level-1 nested collections - and selected a relational pattern based on cardinality, width, identity, application behavior, and access patterns.

Source Characteristic	Preferred Pattern	Reasoning
Small, non-repeating ADT	Pattern A - Inline	Keeps the relationship simple and avoids an unnecessary join
Large 1:1 ADT	Pattern B - Extension	Avoids making the parent excessively wide while preserving 1:1 semantics
Repeating Level-1 collection	Pattern C - Child table	Expresses 1:M cardinality directly in the relational model
Fallback requirement	Pattern D - Transitional sync	Maintains required representations during the controlled fallback window

The signature workflow is: Discover -> Classify -> Analyze Dependencies -> Decide -> Flatten -> Recreate Integrity & Access Paths -> Synchronize -> Test & Reconcile -> Validate -> Cut Over -> Retire.

7. Pattern A - Inline ADT Flattening

A small ADT with a limited number of attributes can often be flattened into conventional columns on the parent table. This is appropriate when the attributes are logically part of the parent, cardinality is 1:1, and the additional columns do not create an undesirable width or sparsity problem.

For example, a fictional ADDRESS_T object containing STREET, CITY, STATE_CODE, and POSTAL_CODE can become four scalar columns. Existing business constraints and null semantics must be evaluated rather than copied mechanically.

The principal advantage is application simplicity: the target model remains a single row without requiring an additional join. The principal risk is excessive parent-table width when the ADT contains many attributes or is sparsely populated.

When ADT attributes are flattened directly into the parent table, the flattened values remain part of the same transactional unit of work as the parent row.

INSERT and UPDATE changes therefore follow the same transaction boundary as the parent operation: the row is committed as a unit or rolled back as a unit.

DELETE does not require separate synchronization of another table for these inline attributes because they are removed with the parent row.

8. Pattern B - One-to-One Extension Flattening

When an ADT contains many attributes, flattening all attributes directly into the parent can make the table unnecessarily wide. A 1:1 extension table can preserve relational clarity while isolating the expanded attributes.

The parent primary key is propagated to the extension table and typically serves as its primary or unique relationship key. The design should explicitly define lifecycle behavior, nullability, referential actions, and whether every parent is expected to have an extension row.

9. Pattern C - One-to-Many Collection Flattening

Repeating Level-1 nested collections are transformed into conventional child tables. The parent primary key is carried into the child as the relationship key. Each child row should also have a stable means of identification. When the source collection does not provide one, the target design may introduce a surrogate identity key while preserving the attributes required for business reconciliation.

This distinction is critical: the parent key alone is not sufficient to identify multiple collection elements. Where no stable source element identifier exists, synchronization must use a documented business/composite match strategy, and an attribute change may be represented operationally as removal of the old combination plus insertion of the new combination.

Duplicate protection should be designed deliberately. Depending on business semantics, this can include a unique constraint on the parent relationship key plus the selected natural-key attributes, together with duplicate-safe INSERT/MERGE logic.

9.1 Illustrative relational target

The following is intentionally simplified and fictional:

```
CREATE TABLE PERSON_CONTACT (
  CONTACT_ID BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  PERSON_ID BIGINT NOT NULL,
  CONTACT_TYPE VARCHAR(30),
  CONTACT_VALUE VARCHAR(255),
  ...
);
```

10. Integrity and Access-Path Preservation

Flattening was more than a structural transformation. CHECK constraints, foreign-key relationships, uniqueness rules, nullability, and indexes associated with the original structures had to be incorporated into the new model or consciously redesigned.

10.1 CHECK constraints

A CHECK constraint expresses a business rule. Its target implementation should preserve the rule even if the physical column location changes.

10.2 Foreign keys

Relationships involving object-relational structures must be mapped to conventional parent/child keys. Referential behavior should be tested for insert, update, and delete paths rather than assumed from DDL alone.

10.3 Indexes

Legacy indexes should not simply be copied by name or column order. The flattened model changes join paths and predicates. Index design should therefore be driven by target queries, relationship keys, selectivity, write cost, and operational workload.

11. Initial Population and Controlled Refresh

Before ongoing synchronization could begin, the flattened structures required an initial population from the Oracle ADT and nested-table representation. In the case study, this one-time refresh was performed during a planned outage so that a known, reconcilable baseline could be established.

A production-scale refresh should be restartable where practical, deduplicate according to defined business rules, record counts and exceptions, and avoid embedding transaction control inside reusable routines that may later be called from triggers.

For large collections, set-based INSERT...SELECT or MERGE patterns are generally preferable to row-by-row processing when the source structure and operational constraints permit them. Exact implementation should be benchmarked against real volume and concurrency.

12. Pattern D - Transitional Fallback Synchronization

The client required an approximately two-week fallback period. During that period, the legacy Oracle representation and the flattened representation had to remain sufficiently synchronized to support controlled fallback. Temporary trigger-based synchronization was therefore treated as transitional architecture, not as a permanent target-state feature.

An important transactional principle emerged: synchronization invoked by a trigger should participate in the caller's transaction. Trigger-called procedures should not independently COMMIT. If the parent DML fails or rolls back, the corresponding flattened changes should roll back with it.

Fallback synchronization is temporary operational architecture. Design it for correctness, observability, and clean retirement.

13. Synchronization Design: Rebuild vs. Incremental

Synchronization can be implemented as a full rebuild or as an incremental comparison. A full rebuild is conceptually simple but can increase redo, locking, and unnecessary writes at scale. Incremental synchronization instead compares the current collection with the flattened representation and applies only the required changes.

For large or frequently updated structures, the preferred pattern is generally incremental synchronization: insert missing rows, remove rows no longer represented, and update or replace changed rows according to the available identity strategy. This reduces unnecessary write amplification while preserving transactional correctness.

Where a collection element had no stable identifier and the object type could not be altered, a composite business key could be used to compare the source collection with the flattened child representation. Under that model, a change to a key attribute is naturally interpreted as delete-old plus insert-new. This behavior must be understood and validated by the application team.

14. Trigger Architecture and Concurrency Lessons

The development work evaluated compound-trigger and separate-trigger approaches. In the observed implementation, compound-trigger synchronization introduced locking/contention concerns. The design was refined to separate AFTER INSERT, AFTER UPDATE, and AFTER DELETE triggers, with reusable synchronization logic centralized where appropriate.

This should not be interpreted as a universal rule that compound triggers are unsuitable. Trigger architecture must be evaluated against the actual Oracle DML pattern, collection behavior, transaction size, concurrency, and locking observed in the environment.

Application DML against flattened columns also required iterative refinement. The available engineering record confirms that application interactions were tested and adjusted, but it does not support publishing a single universal root cause. TP-002 therefore treats this as a testing and compatibility lesson rather than asserting an undocumented mechanism.

15. Testing Became an Engineering Workstream

Conventional migration testing based primarily on row counts was insufficient. Testers and application developers had to understand the source object-relational model, the selected flattening pattern, relationship mappings, and the expected behavior of synchronization and fallback.

Validation covered:

- parent/child cardinality and completeness of flattened collection elements;
- ADT attribute transformation and null semantics;
- surrogate-key generation and parent relationship keys;
- CHECK constraints, foreign keys, uniqueness, and indexes;
- INSERT, UPDATE, and DELETE behavior;
- fallback synchronization and rollback behavior;
- application SQL/DML and dependent database objects; and
- reconciliation between the original Oracle representation and the flattened relational representation.

Row counts are necessary, but they are not sufficient evidence that object-relational semantics were preserved.

16. BeyondX Six-Layer Validation Framework

Layer	Validation Objective
1 - Structural	Expected tables, columns, keys, datatypes, nullability, and mappings exist
2 - Population	Expected parent and child populations are complete
3 - Relationship	Cardinality and parent/child relationships are preserved
4 - Integrity	Constraints, uniqueness, FK behavior, and required access paths behave correctly
5 - Behavioral	Application SQL/DML, triggers, PL/SQL, views, and jobs behave as intended
6 - Operational	Fallback, reconciliation, cutover, monitoring, and retirement controls are proven

This layered approach turns validation into an engineering discipline rather than a final checkpoint. It also makes defects easier to isolate because structural, data, behavioral, and operational failures are not collapsed into a single pass/fail result.

17. Reconciliation at Scale

At multi-terabyte scale, reconciliation should combine aggregate and targeted evidence. Useful techniques include parent counts, expanded collection counts, child counts, null-profile comparisons, key-range sampling, exception queries, duplicate detection, relationship-orphan checks, and business-rule validation.

Where exact full comparisons are expensive, validation can be staged by risk and partition/key range. The goal is not to reduce rigor but to make evidence collection operationally feasible while preserving confidence.

18. Target Platform Considerations

18.1 Amazon Aurora MySQL-Compatible Edition

Aurora MySQL favors conventional relational modeling for these patterns. Identity keys, parent relationship keys, explicit constraints where supported by the chosen table design, and carefully selected indexes provide a portable target. JSON can be useful for genuinely semi-structured requirements, but it should not be used merely to imitate Oracle nested-table storage.

18.2 Amazon Aurora PostgreSQL-Compatible Edition

Aurora PostgreSQL provides arrays, composite types, JSON/JSONB, and rich relational capabilities. These features can model complex data, but they are not direct semantic replacements for Oracle nested tables. The target design should be chosen according to business behavior, portability, query patterns, and operational simplicity rather than superficial feature similarity.

19. Application-Change Minimization

A major objective was to minimize application change while removing Oracle-specific structures. That does not mean preserving every legacy SQL construct. It means selecting a relational representation and transition strategy that preserves required behavior with the smallest reasonable disruption.

Views can sometimes preserve a consumer-facing contract during transition. In other cases, application SQL should be updated explicitly so that the new relational model becomes visible and maintainable rather than hidden behind compatibility layers indefinitely.

20. Cutover, Fallback, and Retirement

The operational sequence used for this class of modernization can be summarized as: schema preparation -> planned outage -> one-time refresh -> reconciliation -> enable fallback synchronization -> application cutover -> controlled fallback window -> retire synchronization -> proceed with cloud migration.

Temporary triggers, compatibility views, staging objects, and synchronization routines should have explicit retirement criteria. A modernization is incomplete if transitional mechanisms become permanent simply because no one owns their removal.

21. Engineering Lessons Learned

- Discovery must include dependent code and operational processes, not only datatype inventory.
- Choose the relational pattern according to cardinality, width, identity, and application behavior.
- For repeating collections, distinguish the parent relationship key from child-row identity.
- When stable child identity is absent, define and document the composite matching strategy.
- Prefer incremental synchronization over unnecessary full child rebuilds when scale and concurrency justify it.
- Keep trigger-called synchronization inside the parent transaction; avoid independent COMMIT behavior.
- Treat fallback as an architecture requirement with a planned end date.
- Test constraints, relationships, DML, dependencies, and operational recovery - not only row counts.
- Retire temporary synchronization and compatibility mechanisms after the fallback requirement expires.

22. BeyondX Engineering Principles

Discovery is not a phase - it is a discipline.

Validation is not a milestone - it is continuous.

Tools accelerate modernization - but they do not replace engineering judgment.

Modernization is successful only when the legacy system can be retired with confidence.

23. Conclusion

Flattening Oracle ADTs and Level-1 nested tables for cloud migration is fundamentally a data-model and dependency-modernization problem. The technical work extends beyond expanding object attributes or copying collection elements into child tables. It requires preserving business semantics, relationships, integrity, access paths, transaction behavior, application compatibility, fallback capability, and evidence of correctness.

The case study demonstrates that this work can be executed at substantial scale when it is approached systematically: discover the structures and dependencies, classify the pattern, design the relational target, preserve integrity, establish a controlled baseline, synchronize only as long as necessary, validate across multiple layers, cut over deliberately, and retire transitional mechanisms.

For organizations moving from Oracle 19c to Amazon Aurora MySQL-Compatible Edition or Amazon Aurora PostgreSQL-Compatible Edition, pre-migration flattening can reduce platform-

specific complexity and make the final migration more predictable. The goal is not to reproduce Oracle internals in another database. The goal is to preserve business meaning in a simpler, supportable, cloud-ready relational model.

Appendix A - Illustrative Synchronization Pattern

The following pseudocode expresses the production-oriented pattern without reproducing client-specific code:

```
AFTER INSERT:
  synchronize(parent_key)

AFTER UPDATE:
  synchronize(parent_key)

AFTER DELETE:
  remove flattened rows for parent_key

synchronize(parent_key):
  insert missing child rows
  remove rows no longer represented
  preserve uniqueness
  do not COMMIT independently
```

Exact Oracle syntax depends on whether the source is a single ADT, a Level-1 collection, the available element identity, schema ownership, privileges, and application DML behavior. Production code should be generated and tested against the actual object definitions rather than copied from a generic example.

Appendix B - Review Checklist

- Inventory all ADTs, collection types, nested columns, and storage structures.
- Identify parent keys and element identity strategy.
- Inventory CHECK constraints, FKs, uniqueness rules, and indexes.
- Inventory triggers, PL/SQL, jobs/events, views, and application SQL/DML.
- Classify each structure as inline, 1:1 extension, or 1:M child-table flattening.
- Define duplicate handling and null semantics.
- Design initial population and reconciliation.
- Design fallback synchronization without independent transaction commits.
- Test INSERT, UPDATE, DELETE, rollback, concurrency, and application behavior.
- Validate relationships and business rules in addition to row counts.
- Define cutover, fallback expiration, and retirement criteria.

Appendix C - Glossary

Term	Meaning
------	---------

ADT	Oracle Abstract Data Type / object type used to group structured attributes
Level-1 nested table	A single-level Oracle collection whose elements are not themselves nested collections in this case study
Flattening	Transformation of object-relational structures into conventional relational columns/tables
Relationship key	Parent key carried into a child table to preserve the parent/child relationship
Surrogate key	System-generated identifier used to uniquely identify a relational row
Fallback synchronization	Temporary mechanism used to keep required representations aligned during a rollback window
Reconciliation	Evidence-based comparison demonstrating that transformed data and relationships are complete and correct